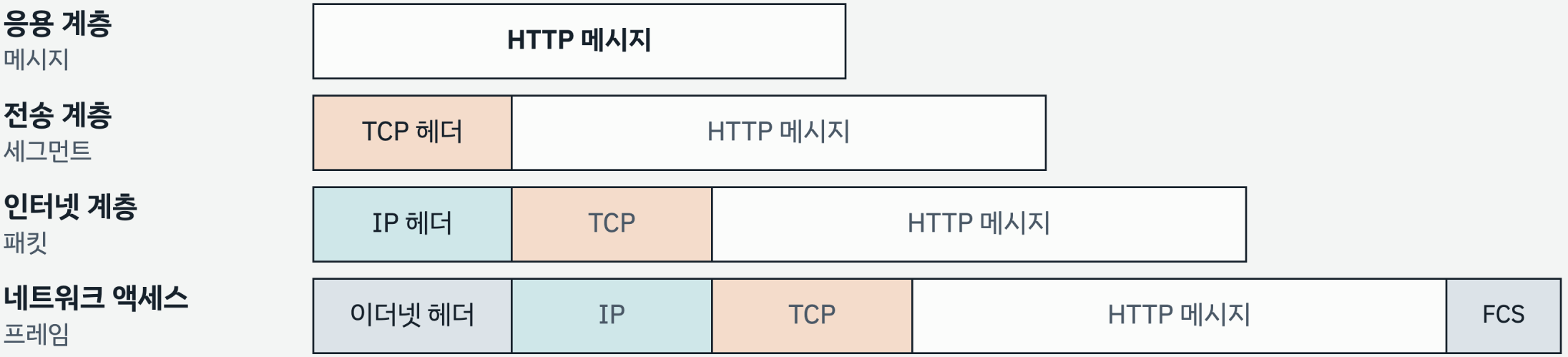
NETWORK STUDY · WEEK 1

캡슐화와 서버 바인딩

HTTP 요청은 어떤 순서로 포장되고,
127.0.0.1에서 돌던 서버는 왜 밖에서 안 보일까

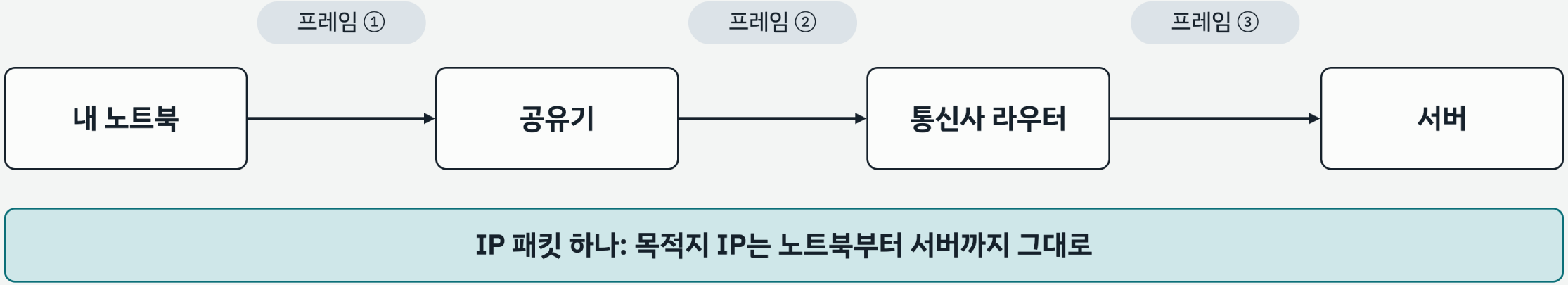
유진영 · 2026년 10월 19일 · 《혼자 공부하는 네트워크》 01-3, 03-2

요청 하나에 헤더가 네 겹 붙는다



보내는 쪽은 위에서 아래로 헤더를 붙이고(캡슐화), 받는 쪽은 아래에서 위로 하나씩 벗긴다(역캡슐화).

가장 짧은 구간을 맡는 계층이 가장 바깥에 붙는다



이더넷 프레임 · 구간마다 새로

장비를 지날 때마다 벗기고 다시 씌운다. MAC 주소는 바로 다음 장비를 가리킨다.

IP 패킷 · 끝까지 하나

라우터는 프레임을 벗겨 IP 헤더를 보고 다음 길을 고른 뒤, 새 프레임에 다시 담는다.

겹마다 가리키는 대상이 다르다

헤더	담긴 주소	가리키는 것	도중에 바뀌나
이더넷	MAC 주소	바로 다음 장비	구간마다 바뀐다
IP	IP 주소	목적지 컴퓨터	목적지는 그대로, 출발지는 NAT에서 바뀜
TCP	포트 번호	그 컴퓨터 안의 프로그램	목적지 포트는 그대로
HTTP	Host · 경로	그 프로그램 안의 리소스	그대로 (HTTPS면 암호화돼 중간에서 못 봄)

바깥 겹일수록 가까운 곳을, 안쪽 겹일수록 먼 곳을 가리킨다.

로드밸런서의 L4·L7이 바로 이 계층 번호다

L4 · 전송 계층

IP와 포트까지만 본다

- TCP 연결 단위로 서버에 나눠 준다
- HTTP 내용은 열어 보지 않아 빠르다
- 예: AWS NLB

L7 · 응용 계층

HTTP 헤더와 경로까지 연다

- /api는 A 서버, /static은 B 서버로 나눌 수 있다
- HTTPS라면 여기서 TLS를 풀어야 한다
- 예: AWS ALB, nginx

번호는 OSI 7계층 기준. 실무 용어에는 TCP/IP 4계층보다 OSI 번호가 더 자주 나온다.

Q04 · 서버 바인딩

127.0.0.1에서 돌던 서버는 왜 밖에서 안 보일까

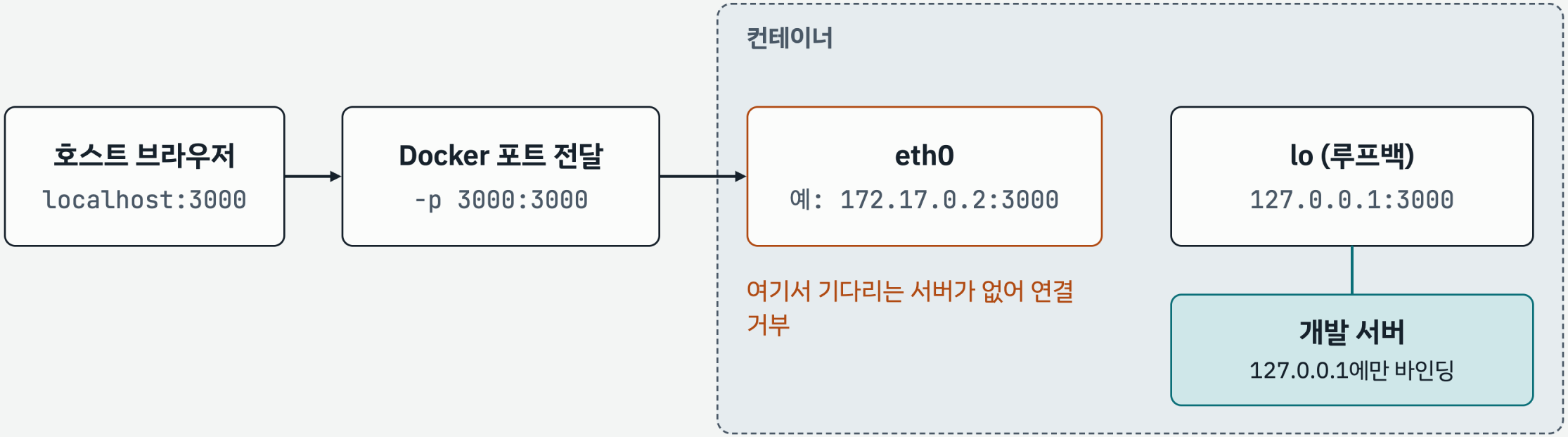
바인딩 주소가 어떤 요청을 받을지 정한다

바인딩 주소	받는 요청	언제 쓰나
127.0.0.1 (localhost)	같은 기기 안에서 온 요청만	혼자 쓰는 개발 서버, 로컬 DB
0.0.0.0	모든 네트워크 인터페이스로 온 요청	컨테이너 안 서버, 다른 기기에서 접속
192.168.0.10 같은 특정 주소	그 인터페이스로 온 요청만	랜카드가 여러 개인 서버

0.0.0.0은 접속하는 주소가 아니다

서버를 띄울 때 "가진 주소 전부에서 기다려라"라는 뜻으로만 쓰는 표기다. 브라우저에는 localhost나 실제 IP를 넣는다.

컨테이너 안의 127.0.0.1은 컨테이너 자신이다



해결: 서버를 0.0.0.0에 바인딩하면 eth0으로 들어온 요청도 받는다

확인하고 고치는 법

지금 어디서 기다리나

```
# macOS
lsof -iTCP -sTCP:LISTEN -n -P
# Linux
ss -ltnp
```

127.0.0.1:3000 → 이 기기 안에서만
***:3000**, **0.0.0.0:3000** → 모든 주소에서

도구별 기본값

도구	기본 바인딩	밖에서 받으려면
Flask	127.0.0.1	flask run --host=0.0.0.0
uvicorn	127.0.0.1	--host 0.0.0.0
Vite	localhost	vite --host
Rails	localhost	rails s -b 0.0.0.0
Node listen(3000)	모든 주소	그대로

0.0.0.0은 누구의 요청이든 받는다는 뜻이다

카페 와이파이의 개발 서버

0.0.0.0으로 띄운 개발 서버나 DB는 같은 와이파이에 붙은 다른 사람도 접속할 수 있다.

Docker -p는 기본이 모든 주소

-p 5432:5432는 호스트의 모든 인터페이스에 포트를 연다. 리눅스에서는 Docker가 방화벽 규칙을 직접 고쳐 ufw 설정을 우회할 수 있다.

로컬 전용 포트는 주소를 적는다

`-p 127.0.0.1:5432:5432`

docker compose의 ports에도 같은 형식으로 쓴다.

정리

1. 헤더는 HTTP → TCP → IP → 이더넷 순으로 붙는다. 구간마다 갈아 끼우는 이더넷이 가장 바깥.
2. 겹마다 가리키는 대상이 다르다. MAC은 다음 장비, IP는 컴퓨터, 포트는 프로그램.
3. 127.0.0.1에 바인딩한 서버는 그 기기 안에서만 보인다. 컨테이너라면 그 컨테이너 안에서만.
4. 0.0.0.0은 편하지만 위험하다. 로컬 전용 포트는 127.0.0.1로 묶는다.

우리 서비스에서는

[docker compose 파일에서 DB·Redis 포트가 127.0.0.1로 묶여 있는지 확인한 결과]